

Exercise 3: Implementing Logging with Spring AOP

LoggingAspect.java

```
package com.library.Aspect;

import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;

@Aspect
public class LoggingAspect {

    @Around("execution(* com.library.Service.*(..))")
    public Object logExecutionTime(ProceedingJoinPoint joinPoint) throws Throwable {

        long start = System.currentTimeMillis();

        Object result = joinPoint.proceed();

        long end = System.currentTimeMillis();

        System.out.println(joinPoint.getSignature().getName()
            + " executed in " + (end - start) + " ms");

        return result;
    }
}
```

BookRepository.java

```
package com.library.Repository;

public class BookRepository {
    public void getBook(){
        System.out.println("getting....");
    }
}
```

BookService.java

```
package com.library.Service;

import com.library.Repository.BookRepository;

public class BookService {
    BookRepository bookRepository;

    public BookService(){

    }

    public void setBookRepository(BookRepository bookrepository){
        this.bookRepository=bookrepository;
    }
    public void know(){
        System.out.println("Geting for you calling repo");
        bookRepository.getBook();
    }
}
```

Main.java

```
package com.library;

import com.library.Service.BookService;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

//TIP To <b>Run</b> code, press <shortcut actionId="Run"/> or
// click the <icon src="AllIcons.Actions.Execute"/> icon in the gutter.
public class Main {
    public static void main(String[] args) {

        ApplicationContext context = new
        ClassPathXmlApplicationContext("applicationContext.xml");
        BookService ser = context.getBean(BookService.class);

        ser.know();
    }
}
```

```
}  
}
```

applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>  
<beans xmlns="http://www.springframework.org/schema/beans"  
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
  xmlns:aop="http://www.springframework.org/schema/aop"  
  xsi:schemaLocation="  
    http://www.springframework.org/schema/beans  
    https://www.springframework.org/schema/beans/spring-beans.xsd  
    http://www.springframework.org/schema/aop  
    https://www.springframework.org/schema/aop/spring-aop.xsd">  
  
  <!-- Repository Bean -->  
  <bean id="bookRepository"  
    class="com.library.Repository.BookRepository"/>  
  <!-- Service Bean -->  
  <bean id="bookService"  
    class="com.library.Service.BookService">  
    <property name="bookRepository"  
      ref="bookRepository"/>  
  </bean>  
  
  <bean id="loggingAspect" class="com.library.Aspect.LoggingAspect"/>  
  <aop:aspectj-autoproxy />  
</beans>
```

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.library</groupId>
  <artifactId>learning_spring</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

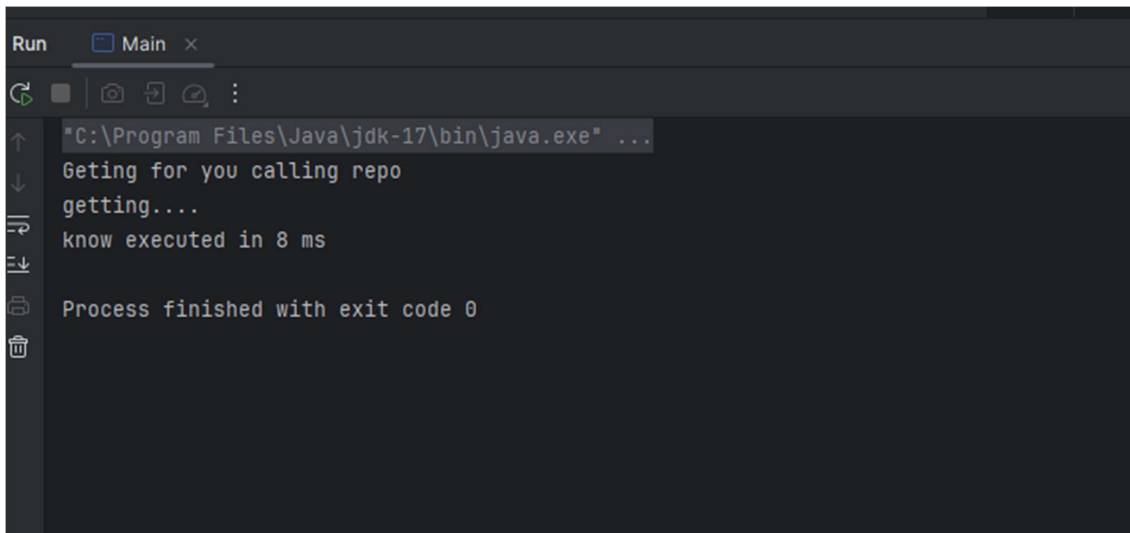
  <dependencies>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-aop</artifactId>
      <version>5.3.39</version>
    </dependency>

    <dependency>
      <groupId>org.aspectj</groupId>
      <artifactId>aspectjweaver</artifactId>
      <version>1.9.22.1</version>
    </dependency>

    <!-- Spring Context -->
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>5.3.39</version>
    </dependency>

  </dependencies>
```

</project>



Exercise 5: Configuring the Spring IoC Container

pom.xml

```
<dependencies>  
  <!-- Spring Context -->  
  <dependency>  
    <groupId>org.springframework</groupId>  
    <artifactId>spring-context</artifactId>  
    <version>5.3.39</version>  
  </dependency>  
</dependencies>
```

applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>  
  <beans xmlns="http://www.springframework.org/schema/beans"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation="
```

<http://www.springframework.org/schema/beans>

<https://www.springframework.org/schema/beans/spring-beans.xsd>>

```
<bean id="bookRepository"
class="com.library.repository.BookRepository"/>
<bean id="bookService"
class="com.library.service.BookService">
<property name="bookRepository"
ref="bookRepository"/>
</bean>
</beans>
```

BookRepository.java

```
package com.library.repository;

public class BookRepository {

    public void displayBook() {

        System.out.println("BookRepository: Fetching book details...");

    }

}
```

BookService.java

```
package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    public void setBookRepository(BookRepository bookRepository) {

        this.bookRepository = bookRepository;

    }

    public void showBook() {

        System.out.println("BookService is working...");

        bookRepository.displayBook();

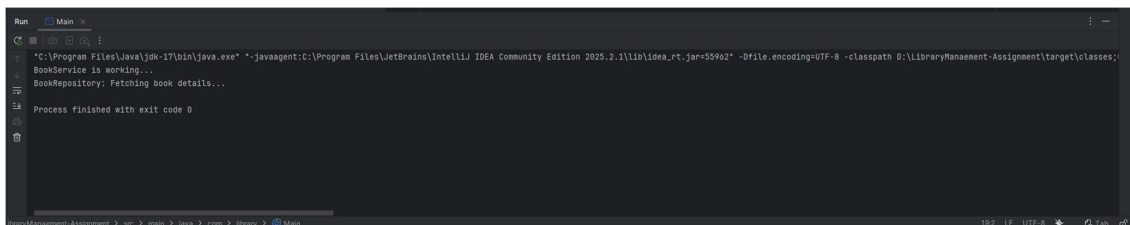
    }

}
```

```
}  
}
```

LibraryManagementApplication.java

```
package com.library;  
  
import com.library.service.BookService;  
  
import org.springframework.context.ApplicationContext;  
  
import org.springframework.context.support.ClassPathXmlApplicationContext;  
  
public class LibraryManagementApplication {  
  
    public static void main(String[] args) {  
  
        ApplicationContext context =  
  
        new ClassPathXmlApplicationContext("applicationContext.xml");  
  
        BookService service =  
  
        context.getBean("bookService", BookService.class);  
  
        service.showBook();  
  
    }  
  
}
```



Exercise 6: Configuring Beans with Annotations

Main.java

```
package com.library;  
  
  
import com.library.Service.BookService;  
  
import org.springframework.context.ApplicationContext;
```

```

import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Main {

    public static void main(String[] args) {

        ApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");

        BookService service = context.getBean(BookService.class);

        service.know();
    }
}

```

applicationContext.xml

```

<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        https://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/context
        https://www.springframework.org/schema/context/spring-context.xsd">

    <context:component-scan base-package="com.library"/>

</beans>

```

BookService.java

```

package com.library.Service;

import com.library.Repository.BookRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service

```



```

public class BookService {

    @Autowired
    private BookRepository bookRepository;

    public void know() {
        System.out.println("Getting for you calling repo");
        bookRepository.getBook();
    }
}

```

BookRepository.java

```

package com.library.Repository;

import org.springframework.stereotype.Repository;

@Repository
public class BookRepository {

    public void getBook() {
        System.out.println("getting....");
    }
}

```

pom.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.library</groupId>
    <artifactId>learning_spring</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>1.8</maven.compiler.source>

```

```
<maven.compiler.target>1.8</maven.compiler.target>
<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>
```

```
<dependencies>
```

```
<!-- Spring Context -->
```

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>5.3.39</version>
</dependency>
```

```
<!-- Spring AOP -->
```

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-aop</artifactId>
  <version>5.3.39</version>
</dependency>
```

```
<!-- AspectJ Weaver -->
```

```
<dependency>
  <groupId>org.aspectj</groupId>
  <artifactId>aspectjweaver</artifactId>
  <version>1.9.22.1</version>
</dependency>
```

```
<!-- Spring WebMVC -->
```

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-webmvc</artifactId>
  <version>5.3.39</version>
</dependency>
```

```
</dependencies>
```

```
<build>
```

```
<plugins>
```

```
<plugin>
```

```
<groupId>org.apache.maven.plugins</groupId>
```

```

        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.11.0</version>

        <configuration>
            <source>1.8</source>
            <target>1.8</target>
        </configuration>

    </plugin>

</plugins>
</build>

</project>

```

Exercise 7: Implementing Constructor and Setter Injection

Constructor Injection

applicationContext.xml

```

<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           https://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.springframework.org/schema/context
           https://www.springframework.org/schema/context/spring-context.xsd">

    <bean id="bookRepository"
          class="com.library.Repository.BookRepository"/>

    <bean id="bookService"
          class="com.library.Service.BookService">

        <constructor-arg ref="bookRepsitory" />

    </bean>

</beans>

```

BookService.java

```
package com.library.Service;

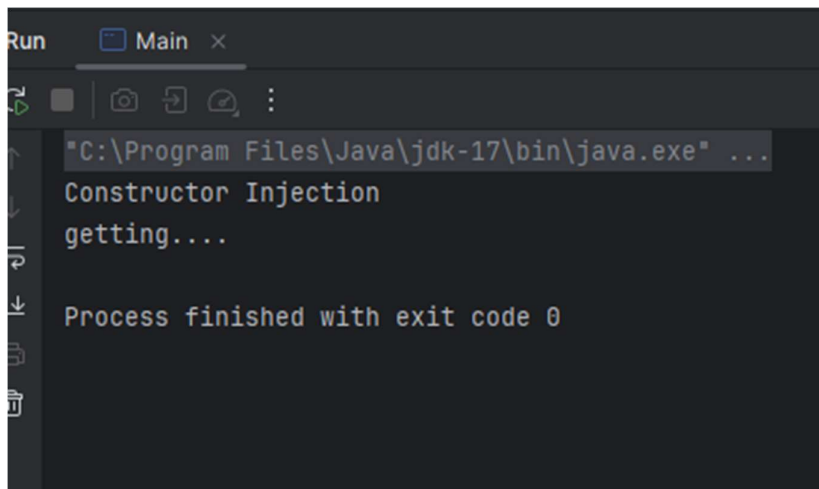
import com.library.Repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    public BookService(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void know() {
        System.out.println("Constructor Injection");
        bookRepository.getBook();
    }
}
```



Setter Injection

applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```

xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="
http://www.springframework.org/schema/beans
https://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
https://www.springframework.org/schema/context/spring-context.xsd">

<bean id="bookRepository"
    class="com.library.Repository.BookRepository"/>

<bean id="bookService"
    class="com.library.Service.BookService">

    <property name="bookRepository" ref="bookRepository"/>

</bean>

</beans>

```

BookService.java

```

package com.library.Service;

import com.library.Repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

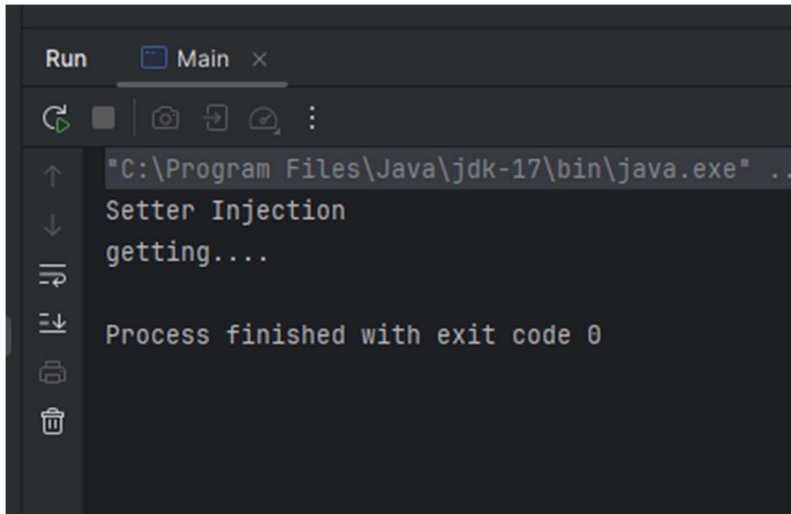
    public BookService() {
    }

    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void know() {
        System.out.println("Setter Injection");
        bookRepository.getBook();
    }
}

```

```
}  
}
```



Exercise 8: Implementing Basic AOP with Spring

applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<beans xmlns="http://www.springframework.org/schema/beans"  
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
  xmlns:aop="http://www.springframework.org/schema/aop"  
  xsi:schemaLocation="  
    http://www.springframework.org/schema/beans  
    https://www.springframework.org/schema/beans/spring-beans.xsd  
    http://www.springframework.org/schema/aop  
    https://www.springframework.org/schema/aop/spring-aop.xsd">
```

```
  <bean id="bookRepository"  
    class="com.library.Repository.BookRepository"/>
```

```
  <bean id="bookService"  
    class="com.library.Service.BookService">  
    <property name="bookRepository" ref="bookRepository"/>  
  </bean>
```

```
<bean id="loggingAspect"
      class="com.library.Aspect.LoggingAspect"/>

<aop:aspectj-autoproxy/>

</beans>
```

LoggingAspect.java

```
package com.library.Aspect;

import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.aspectj.lang.annotation.After;

@Aspect
public class LoggingAspect {

    @Before("execution(* com.library.Service.*(..))")
    public void beforeMethod() {
        System.out.println("Before method execution...");
    }

    @After("execution(* com.library.Service.*(..))")
    public void afterMethod() {
        System.out.println("After method execution...");
    }
}
```

Exercise 9: Creating a Spring Boot Application

DemoApplication.java

```
package com.libraryManagement.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {
```

```
    public static void main(String[] args) {  
        SpringApplication.run(DemoApplication.class, args);  
    }  
  
}
```

BookRepository.java

```
package com.libraryManagement.demo.Repository;
```

```
import org.springframework.data.jpa.repository.JpaRepository;  
import com.libraryManagement.demo.Entity.Book;
```

```
public interface BookRepository extends JpaRepository<Book, Long> {  
  
}
```

Book.java

```
package com.libraryManagement.demo.Entity;
```

```
import jakarta.persistence.Entity;  
import jakarta.persistence.GeneratedValue;  
import jakarta.persistence.GenerationType;  
import jakarta.persistence.Id;
```

```
@Entity
```

```
public class Book {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    private String title;
```

```
    private String author;
```

```
    public Book() {
```



```
}

public Book(Long id, String title, String author) {
    this.id = id;
    this.title = title;
    this.author = author;
}

public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public String getTitle() {
    return title;
}

public void setTitle(String title) {
    this.title = title;
}

public String getAuthor() {
    return author;
}

public void setAuthor(String author) {
    this.author = author;
}
}
```

BookController.java

```
package com.libraryManagement.demo.Controller;
```

```
import com.libraryManagement.demo.Entity.Book;
import com.libraryManagement.demo.Repository.BookRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
```

```
import java.util.List;
```

```
@RestController
@RequestMapping("/books")
public class BookController {
```

```
    @Autowired
    private BookRepository repository;
```

```
    @PostMapping
    public Book addBook(@RequestBody Book book) {
        return repository.save(book);
    }
```

```
    @GetMapping
    public List<Book> getAllBooks() {
        return repository.findAll();
    }
```

```
    @GetMapping("/{id}")
    public Book getBook(@PathVariable Long id) {
        return repository.findById(id).orElse(null);
    }
```

```
    @PutMapping("/{id}")
    public Book updateBook(@PathVariable Long id,
        @RequestBody Book book) {
```

```
        Book b = repository.findById(id).orElse(null);
```

```
        if (b != null) {
            b.setTitle(book.getTitle());
            b.setAuthor(book.getAuthor());
            return repository.save(b);
        }
```

```
        return null;
    }

    @DeleteMapping("/{id}")
    public String deleteBook(@PathVariable Long id) {

        repository.deleteById(id);

        return "Book Deleted Successfully";
    }
}
```

HTTP **http://localhost:8081/books**

POST **http://localhost:8081/books**

Params Authorization Headers (8) **Body** Pre-request Script Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary **JSON**

```
1 {
2   "title": "Spring Boot",
3   "author": "John"
4 }
```

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize **JSON**

```
1 {
2   "id": 1,
3   "title": "Spring Boot",
4   "author": "John"
5 }
```

HTTP

http://localhost:8081/books

GET

http://localhost:8081/books

Params

Authorization

Headers (6)

Body

Pre-request Script

Tests

Settings

☐ none

☒ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

JSON

▼

1

body

Cookies

Headers (5)

Test Results

Pretty

Raw

Preview

Visualize

JSON

▼

1

2

3

4

5

6

7

{

"author": "John"

"title": "Spring Boot",

"id": 1,

}

